

Este curso se enfocó en paralelización con openmp y openacc y fue impartido por el profesor Juan Carlos Cajas García (¡gracias profesor!).

Resumen

En este informe se explica el funcionamiento del código `calor2D.f90`, el cual calcula numéricamente la solución estacionaria de la ecuación de calor con condiciones de frontera de Dirichlet. El caso de una dimensión (1D) sólo puede llevarse a cabo de forma secuencial usando un método de solución exacta a la discretización de la ecuación de difusión en 1D conocido como el algoritmo de Thomas, mientras que para 2D se utilizó nuevamente dicho algoritmo en una rutina para ejecutarlo de forma paralelizada usando `openmp` repartiéndolo en las diferentes líneas en las direcciones x y y . Este informe se enfoca en comprender el algoritmo, su implementación en Fortran usando `openmp` así como los resultados y el rendimiento obtenido mediante la paralelización utilizando diferentes números de hilos simultáneamente.

Índice

1. Introducción

2. Caso 1D

- 2.1. Discretización
- 2.2. El algoritmo de Thomas
- 2.3. El programa `calor1D.f90` y el módulo `tridiagonal.f90` . .

3. Caso 2D

- 3.1. Discretización
- 3.2. Convergencia
- 3.3. OpenMP
- 3.4. El programa `calor2D.f90`

4. Resultados

- 4.1. Solución estacionaria
- 4.2. Rendimiento

5. Conclusiones

A. Programa `mod_utiles.f90`

1. Introducción

La ecuación de difusión de calor

$$\frac{\partial T}{\partial t} = D \nabla^2 T$$

es un ejemplo paradigmático de un modelo físico sencillo. Poder resolverlo numéricamente es un buen antecedente para poder manejar las herramientas necesarias para hacer simulaciones de fenómenos físicos más complicados; por ejemplo, para implementar un método numérico de solución a las ecuaciones de Navier-Stokes, la cual, ha encontrado infinitas aplicaciones en varias disciplinas.

En este texto se exploran los casos de 1 y 2 dimensiones resolviendo únicamente la solución estacionaria. Es decir, imponiendo la ecuación de Laplace

$$\nabla^2 T = 0.$$

2. Caso 1D

2.1. Discretización

En régimen estacionario y sin fuentes, la ecuación de calor en una dimensión se reduce a resolver la ecuación diferencial $T''(x) = 0$ sobre el intervalo $[0, L]$, con condiciones de Dirichlet $T(0) = T_0$ y $T(L) = T_L$.

Discretizando el dominio en N nodos con espaciamento $\Delta x = L/(N - 1)$ y aproximando la segunda derivada por su diferencia centrada

$$\frac{d^2 T}{dx^2} \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{\Delta x^2}.$$

se obtiene que cada nodo interior queda ligado únicamente a sus vecinos inmediatos

$$1 \quad \frac{T_{i+1} - 2T_i + T_{i-1}}{(\Delta x)^2} = 0, \quad i = 1, \dots, N - 2. \quad (1)$$

1 Multiplicando por $(\Delta x)^2$, la ecuación se simplifica a

$$1 \quad T_{i-1} - 2T_i + T_{i+1} = 0, \quad i = 1, \dots, N - 2. \quad (2)$$

Reuniendo (1) con las dos filas referentes a las condiciones de frontera, el problema se convierte en un sistema lineal $A \mathbf{T} = \mathbf{d}$ cuya matriz es *tridiagonal*: solo son no nulas la diagonal principal b_i y las dos adyacentes a_i (inferior) y c_i (superior),

$$2 \quad \begin{pmatrix} b_0 & c_0 & & & \\ a_1 & b_1 & c_1 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{N-1} & b_{N-1} \end{pmatrix} \mathbf{T} = \mathbf{d}, \quad (3)$$

donde $a_i = c_i = 1$, $b_i = -2$ y $d_i = 0$ para $1 \leq i < N - 1$. Para las condiciones de Dirichlet en ambos puntos de la frontera $b_0 = b_{N-1} = 1$, $a_0 = a_{N-1} = 0$, $d_0 = T_0$ y $d_{N-1} = T_L$.

5 Como curiosidad, si se deseara implementar una condición de Neumann (por una derivada conocida $T'(0) = q$ en el lado izquierdo), basta utilizar una diferencia hacia adelante $\frac{T_1 - T_0}{\Delta x} = q$. Al recomodar multiplicando por Δx se llega a la forma $-T_0 + T_1 = q\Delta x$, con lo cual se asignarían libremente los componentes de la matriz a $b_0 = -1$, $c_0 = 1$ y $d_0 = q\Delta x$.

2.2. El algoritmo de Thomas

La estructura diagonal de la matriz en (3) permite resolver el sistema de ecuaciones sin una eliminación gaussiana completa. El algoritmo de Thomas lo hace en dos barridos. En el barrido hacia adelante se eliminan los coeficientes inferiores (partiendo de $c'_0 = c_0/b_0$ y $d'_0 = d_0/b_0$),

$$c'_i = \frac{c_i}{b_i - a_i c'_{i-1}}, \quad d'_i = \frac{d_i - a_i d'_{i-1}}{b_i - a_i c'_{i-1}},$$

y en la sustitución hacia atrás se recupera la solución,

$$T_{N-1} = d'_{N-1}, \quad T_i = d'_i - c'_i T_{i+1}.$$

Su costo es $O(N)$, frente al $O(N^3)$ de una eliminación general, lo que hace de Thomas el método natural para la matriz tridiagonal que produce el esquema de diferencias finitas en 1D¹.

2.3. El programa `calor1D.f90` secuencial y el módulo `tridiagonal.f90`

El módulo `tridiagonal.f90` contiene una implementación del algoritmo de Thomas en Fortran. Este méto, como se mencionó en el apartado anterior, aprovecha que la matriz del problema solo tiene elementos distintos de cero en la diagonal principal y sus diagonales vecinas. Primero elimina los términos inferiores de la matriz y después calcula la solución mediante sustitución hacia atrás, obteniendo el perfil de temperatura de manera rápida y eficiente.

¹Se puede consultar una explicación más completa de este algoritmo y su pseudocódigo en la página de wikipedia [Tridiagonal matrix algorithm](#).

```

1 subroutine tri(a,b,c,r,u,n)
2  !
3  ! Subrutina que resuelve el problema
4  !
5  ! Ax=r donde A es una matriz tridiagonal con
6  ! elementos a, b, c
7  ! y devuelve el valor de u
8  !
9  implicit none
10 !
11 integer,          intent(in)    :: n
12 double precision, intent(in)    :: a(n),c(n)
13 double precision, intent(inout) :: b(n),r(n)
14 double precision, intent(inout) :: u(n)
15 integer :: i
16
17 u=0.d0
18 ! eliminacion elementos bajo la matriz
19 gausselim: do i=2,n
20     r(i)=r(i)-(a(i)/b(i-1))*r(i-1)
21     b(i)=b(i)-(a(i)/b(i-1))*c(i-1)
22 end do gausselim
23 !
24 ! solucion para u
25 !
26 u(n)=r(n)/b(n)
27 sustatras: do i=n-1,1,-1
28     u(i)=(r(i)-c(i)*u(i+1))/b(i)
29 end do sustatras
30 !
31 end subroutine tri

```

El código `calor1D.f90` simplemente construye los arreglos 1D para el sistema. Cabe notar que establece las condiciones de frontera de la siguiente manera: del lado izquierdo impone una derivada igual a -1 , y del lado derecho fija $T_L = 0$. Una vez preparado, llama a la subrutina en `tridiagonal.f90` para resolver todo secuencialmente con el algoritmo de Thomas.

```

1 Program Calor1D
2  !
3  Implicit none
4  !
5  ! Declaracion de variables
6  !
7  ! Iteradores y tamaño del problema
8  !
9  integer :: ii
10 integer, parameter :: nn = 61
11 !
12 ! Variables del dominio computacional
13 !
14 double precision :: ll, deltax
15 !
16 ! Variables del problema fisico
17 !
18 double precision :: xx(nn)    ! variable de la malla
19 ! double precision :: aa(nn,nn) ! matriz para el problema
20 double precision :: tt(nn)    ! vector de incognitas
21 double precision :: rr(nn)    ! vector de resultados del s. ecuaciones
22 double precision :: cf(2)     ! vector de condiciones de frontera
23 !
24 double precision :: aa(nn), bb(nn), cc(nn) ! Variables para almacenar
25 !                                           ! matriz tridiagonal sobredimensionada
26 !
27 ! Dominio computacional
28 !
29 ll      = 10.d0
30 deltax  = 10.d0/dfloat(nn-1)
31 ! Inicializacion de variables
32 !
33 do ii = 1, nn
34     xx(ii) = (ii - 1) * deltax
35 end do
36 aa(:)    = 0.d0
37 bb(:)    = 0.d0
38 cc(:)    = 0.d0
39 tt(:)    = -1.d0
40 rr(:)    = 0.d0
41 !
42 ! Condiciones de frontera
43 !
44 cf(1) = -1.d0
45 cf(2) = 0.d0
46 !
47 ! Ensamblar la matriz
48 !
49 do ii = 2, nn-1
50     !
51     aa(ii) = 1.d0
52     bb(ii) = -2.d0
53     cc(ii) = 1.d0
54     rr(ii) = 0.d0
55     !
56 end do
57 !
58 ! Impone cond. frontera
59 !
60 aa(1)    = 0.d0 ! no se usa en los c'alculos
61 bb(1)    = -1.d0
62 cc(1)    = 1.d0
63 rr(1)    = cf(1) * deltax
64 !
65 aa(nn)   = 0.d0
66 bb(nn)   = 1.d0
67 cc(nn)   = 0.d0 ! no se usa en los c'alculos
68 rr(nn)   = cf(2)
69 !
70 ! Resolver el problema algebraico
71 !
72 call tri(aa,bb,cc,rr,tt,nn)
73 !
74 ! Postproceso
75 !
76 do ii = 1, nn
77     write(*,*) xx(ii), tt(ii)
78 end do
79 end Program Calor1D

```

Para compilar estos dos códigos y ejecutar el programa se pueden correr en la terminal los siguientes comandos

```

gfortran tridiagonal.f90 calor1D.f90  -o calor1D
./calor1D

```

y el resultado obtenido al ejecutar el programa `calor1D` es la impresión de cada punto en la malla guardado en `xx(-)` y su respectivo valor de temperatura que fue calculado y almacenado en `tt(-)`. En este caso, si se grafican los valores, se obtiene una línea recta con pendiente $T'(x) \equiv -1$ y tal que $T(x = 10.0) = 0$ (por lo tanto $T(x = 0) = 10.0$ salvo que con pequeños errores debido a la aritmética de punto flotante).

3. Caso 2D

3.1. Discretización

En dos dimensiones, la ecuación de Laplace es

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0,$$

sobre un dominio rectangular $[0, L_x] \times [0, L_y]$ con condiciones de Dirichlet en todas las fronteras. Discretizando con N_x nodos en x y N_y nodos en y , con espaciamentos $\Delta x = L_x/N_x$ y $\Delta y = L_y/N_y$, y aproximando cada segunda derivada por diferencias centradas, la ecuación evaluada en el nodo interior (i, j) queda

$$\frac{T_{i-1,j} - 2T_{i,j} + T_{i+1,j}}{(\Delta x)^2} + \frac{T_{i,j-1} - 2T_{i,j} + T_{i,j+1}}{(\Delta y)^2} = 0. \quad (4)$$

A diferencia del caso 1D, aquí los nodos en x están acoplados con los de y , por lo que el sistema resultante ya no es tridiagonal en el sentido estricto: es un sistema disperso de $N_x N_y$ incógnitas que no puede resolverse directamente con el algoritmo de Thomas.

Método de líneas (ADI simplificado)

Para conservar la eficiencia del algoritmo de Thomas, se recurre a un método iterativo por *barridos alternos de líneas*. La idea es fijar temporalmente una dirección y resolver la otra como un sistema tridiagonal 1D. Partiendo de (4) y reagrupando,

- **Barrido en x :** para cada fila j fija, los valores $T_{i,j\pm 1}$ se tratan como conocidos y se pasan al lado derecho. La ecuación en el nodo (i, j) adopta la forma

$$-\frac{T_{i,j-1} + T_{i,j+1}}{(\Delta y)^2} = \frac{1}{(\Delta x)^2} T_{i-1,j} - 2\left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}\right) T_{i,j} + \frac{1}{(\Delta x)^2} T_{i+1,j}, \quad (5)$$

que es un sistema tridiagonal de tamaño N_x por cada j .

- **Barrido en y :** para cada columna i fija, los valores $T_{i\pm 1,j}$ se tratan como conocidos y se pasan al lado derecho. La ecuación en (i, j) es

$$-\frac{T_{i-1,j} + T_{i+1,j}}{(\Delta x)^2} = \frac{1}{(\Delta y)^2} T_{i,j-1} - 2\left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}\right) T_{i,j} + \frac{1}{(\Delta y)^2} T_{i,j+1}, \quad (6)$$

que es un sistema tridiagonal de tamaño N_y por cada i .

3.2. Convergencia

Una condición de convergencia se puede implementar en los dos barridos mencionados en el apartado anterior si se repiten iterativamente hasta que la norma euclidiana de la diferencia entre iteraciones sucesivas cae por debajo de una tolerancia ε ,

$$\|T^{(k+1)} - T^{(k)}\|_2 < \varepsilon.$$

Cada uno de los sistemas tridiagonales se resuelve con el algoritmo de Thomas.

3.3. OpenMP

OpenMP (Open Multi-Processing) es una interfaz de programación de aplicaciones (API) diseñada para el desarrollo de software en paralelo en arquitecturas de memoria compartida, compatible con los lenguajes C, C++ y Fortran (en nuestro curso, vimos únicamente cómo usarlo en Fortran). El modelo de ejecución se basa en una estructura secuencial-paralela en la que el programa inicia con un único hilo conductor, denominado hilo maestro. Al

alcanzar una región identificada como paralela, dicho hilo genera un conjunto de hilos de soporte que ejecutan las tareas simultáneamente, integrándose nuevamente al hilo principal una vez concluida la operación.

La principal ventaja de OpenMP radica en su capacidad para abstraer la complejidad de bajo nivel en la gestión de hilos, permitiendo implementar paralelismo mediante directivas de compilador sencillas (conocidas como *pragmas*), tales como `!$omp parallel do`. Asimismo, el sistema ofrece mecanismos precisos para delimitar el alcance de las variables en ámbitos públicos o privados, garantizando así un control eficiente sobre las condiciones de carrera y la sincronización. Para su implementación, se requiere la inclusión de la librería de cabecera `omp.h` y la activación de la bandera `-fopenmp` durante el proceso de compilación².

3.4. El programa calor2D.f90 paralelizado con openmp

A continuación presentamos y explicamos el código en el que se implementa la discretización y la condición de paro que se explicó anteriormente. En el apéndice A se colocó el código del módulo `utiles`, escrito en el programa `mod_utiles.f90`, en el cual se especifica el número de nodos en cada dirección ($N_x = 120, N_y = 60$) y el máximo de iteraciones permitidas (1000 iteraciones).

```
1 Program Calor2D
2 !
3 use omp_lib
4 !
5 use utiles, only : postproceso_vtk
6 use utiles, only : residuo_temp
7 use utiles, only : nx, ny, itermax
8 !
9 Implicit none
10 !
11 ! Declaracion de variables
12 !
13 ! Iteradores y tamaño del problema
14 !
15 integer :: ii, jj, iter
16 !
17 ! Variables del dominio computacional
18 !
19 double precision :: lx, ly, deltax, deltay
20 !
21 ! Variable para el residuo de iteraciones, y tolerancia
22 !
23 double precision :: residuo, tolerancia
24 !
25 ! Variables del problema fisico
26 !
27 double precision :: xx(nx), yy(ny) ! variables de la malla
28 double precision :: tt(nx,ny,2) ! vector de incognitas, 1 para la iteraci'on
    ⇐ actual y 2 para la anterior
29 double precision :: resid_tt(nx,ny) ! vector de residuo
30 double precision :: tx(nx), ty(ny) ! vectores de incognitas 1D
31 double precision :: rx(nx),ry(ny) ! vector de resultados del s. ecuaciones
32 double precision :: cfx(ny,2),cfy(nx,2) ! vector de condiciones de frontera
33 !
34 double precision :: ax(nx), bx(nx), cx(nx) ! Variables para almacenar
35 ! ! matriz tridiagonal sobredimensionada
36 !
37 double precision :: ay(ny), by(ny), cy(ny) ! Variables para almacenar
38 ! ! matriz tridiagonal sobredimensionada
39 !
40 ! Variables de postproceso
41 !
42 character(48) :: archivo
43 !
44 ! Tolerancia
45 !
46 tolerancia = 1d-4
47 !
48 ! Dominio computacional
49 !
50 lx = 10.d0
51 deltax = lx/nx
52 ly = 5.d0
53 deltay = ly/ny
54 do jj = 1, ny
55     yy(jj)=(jj-1)*deltay
56 end do
57 do ii = 1, nx
58     xx(ii)=(ii-1)*deltax
59 end do
60 !
61 ! Inicializacion de variables
62 !
63 !  $\bar{x}(:) = 0.d0$ 
64 ax(:) = 0.d0
65 bx(:) = 0.d0
66 cx(:) = 0.d0
67 rx(:) = 0.d0
68 ay(:) = 0.d0
69 by(:) = 0.d0
70 cy(:) = 0.d0
71 ry(:) = 0.d0
72 tt(:, :, :) = 0.d0
73 !
74 ! Condiciones de frontera en direcci'on x
75 !
76 ! Estos bucles pueden paralelizarse, pero hay que valorar bien la cantidad
77 ! de trabajo que se va a compartir respecto al tiempo necesario para abrir
78 ! los hilos paralelos.
79 !
80 do ii = 1, ny
81     cfx(ii,1) = 1.d0
82     cfx(ii,2) = 0.d0
83 end do
84 !
85 ! Condiciones de frontera en direcci'on y
86 !
```

```
87 do jj = 1, nx
88     cfy(jj,1) = 1.d0
89     cfy(jj,2) = 0.d0
90 end do
91 !
92 bucle_iteraciones: do iter = 1, itermax
93 !
94 ! Inicializamos el valor de la iteraci'on anterior
95 !$omp parallel do
96 do jj = 2, ny-1
97     do ii = 2, nx-1
98         tt(ii,jj,2) = tt(ii,jj,1)
99     end do
100 end do
101 !$omp end parallel do
102 !
103 !-----
104 !
105 ! Paralelizamos el barrido en la direcci'on y en bandas
106 ! compuestas por grupos de l'ineas, observamos que si usamos pocas
107 ! l'ineas tenemos una aceleraci'on pobre o ausente
108 !
109 !$omp parallel do default(none) &
110 !$omp shared( deltax, deltay, tt, cfx) &
111 !$omp private( ax, bx, cx, rx, tx )
112 barrido_y: do jj = 2, ny-1
113 !
114 ! Es posible combinar directivas de openmp, por ejemplo,
115 ! si necesitamos una regi'on paralela unicamente para un bucle,
116 ! podemos combinar "parallel" con "do"
117 !
118 ensambla_tri_x: do ii = 2, nx-1
119
120     ax(ii) = 1.d0/(deltax*deltax)
121     bx(ii) =-2.d0*(1.d0/(deltax*deltax)+ 1.d0/(deltay*deltay))
122     cx(ii) = 1.d0/(deltax*deltax)
123     rx(ii) =-1.d0/(deltay*deltay)*tt(ii,jj-1,1)-1.d0/(deltay*deltay)&
124         & * tt(ii,jj+1,1)
125
126 end do ensambla_tri_x
127 !
128 ! Impone cond. frontera
129 !
130 ax(1) = 0.d0 ! no se usa en los c'alculos
131 bx(1) = 1.d0
132 cx(1) = 0.d0
133 rx(1) = cfx(jj,1)
134 !
135 ax(nx) = 0.d0
136 bx(nx) = 1.d0
137 cx(nx) = 0.d0 ! no se usa en los c'alculos
138 rx(nx) = cfx(jj,2)
139 !
140 ! Resolver el problema algebraico
141 !
142 call tri(ax,bx,cx,rx,tx,nx)
143 !
144 ! Actualizar la temperatura de la placa
145 !
146 do ii = 1, nx
147
148     tt(ii,jj,1) = tx(ii)
149
150 end do
151 !
152 end do barrido_y
153 !$omp end parallel do
154 !
155 !-----
156 !
157 ! Paralelizamos el barrido en la direcci'on x en bandas
158 ! compuestas por grupos de l'ineas, observamos que si usamos pocas
159 ! l'ineas tenemos una aceleraci'on pobre o ausente
160 !
161 !$omp parallel do default(none) &
162 !$omp shared( deltax, deltay, tt, cfy) &
163 !$omp private( ay, by, cy, ry, ty )
164 barrido_x: do ii = 2, nx-1
165
166     ensambla_tri_y: do jj = 2, ny-1
167
168         ay(jj) = 1.d0/(deltay*deltay)
169         by(jj) =-2.d0*(1.d0/(deltax*deltax)+ 1.d0/(deltay*deltay))
170         cy(jj) = 1.d0/(deltay*deltay)
171         ry(jj) =-1.d0/(deltax*deltax)*tt(ii-1,jj,1)-1.d0/(deltax*deltax)*&
172             tt(ii+1,jj,1)
173
174 end do ensambla_tri_y
175 !
176 ! Impone cond. frontera
177 !
178 ay(1) = 0.d0 ! no se usa en los c'alculos
179 by(1) = 1.d0
180 cy(1) = 0.d0
181 ry(1) = cfy(ii,1)
182 !
183 ay(ny) =-1.d0
184 by(ny) = 1.d0
185 cy(ny) = 0.d0 ! no se usa en los c'alculos
186 ry(ny) = cfy(ii,2)
187 !
188 ! Resolver el problema algebraico
189 !
190 call tri(ay,by,cy,ry,ty,ny)
191 !
192 !
193 ! Actualizar la temperatura de la placa
194 !
195 do jj = 1, ny
196
197     tt(ii,jj,1) = ty(jj)
198
199 end do
200 !
201 end do barrido_x
202 !$omp end parallel do
203 !
204 ! Criterio de convergencia
205 !
206 residuo = 0.d0
207 !$omp parallel do reduction(+:residuo)
208 do ii = 2, nx-1
209     do jj = 2, ny-1
```

²Para una introducción amena a OpenMP se recomienda consultar la referencia [1].


```
211         residuo = residuo + (tt(ii,jj,1)-tt(ii,jj,2))*(tt(ii,jj,1)-tt(ii,jj,2))
212
213     end do
214 end do
215 !$omp end parallel do
216 !
217 residuo = sqrt(residuo)
218 !
219 ! write(*,*) "DEBUG: ", iter, residuo
220 !
221 ! if( residuo < tolerancia )exit
222 !
223 end do bucle_iteraciones
224 !
225 write(*,*) "Convergencia en ", iter, " iteraciones"
226 !
227 call residuo_temp( tt(1:nx,1:ny,1), deltax, deltay, resid_tt )
228 !
229 ! Aunque es muy tentador, no podemos paralelizar este bucle,
230 ! el archivo queda desordenado y gnuplot (y otros graficadores) no
231 ! los procesan bien.
232 !
233 archivo = 'salida.vtk'
234 !
235 call postproceso_vtk(xx,yy,tt(1:nx,1:ny,1), resid_tt ,archivo)
236 !
237 end Program Calor2D
```

En la línea 3 se añade `use omp_lib`, lo que permite al programa acceder a las funciones y directivas de OpenMP.

En las líneas 76 a 83 y 87 a 90 se inicializan las condiciones de frontera. Como se menciona en los comentarios del código, estos bucles no se paralelizan debido a que el volumen de trabajo es mínimo; el costo computacional (*overhead*) de crear y gestionar los hilos superaría el beneficio de la ejecución paralela en un arreglo tan pequeño.

Sincronización en la actualización del estado (Líneas 95–101)

Al inicio de cada paso en el bucle temporal, se realiza una copia del estado actual de la temperatura al arreglo de la iteración anterior (`tt(:, :, 2)`). Esto se paraleliza mediante la directiva `!$omp parallel do`. Dado que no existen dependencias de datos entre celdas distintas para esta operación de asignación, los hilos de ejecución distribuyen las filas de la matriz de manera independiente y segura.

Barridos de líneas y alcance de variables (Líneas 109–153 y 161–202)

En la línea 3 se añade `use omp_lib` para dar acceso a las funciones de OpenMP.

Las líneas 76–83 y 87–90 inicializan las condiciones de frontera sin paralelismo, ya que el arreglo es tan pequeño que el costo de gestionar los hilos superaría cualquier ganancia.

Copia del estado anterior (Líneas 95–101)

Al inicio de cada paso temporal se copia la temperatura actual al arreglo `tt(:, :, 2)`. Esta operación no tiene dependencias entre celdas, así que se paraleliza con `!$omp parallel do` y cada hilo trabaja sobre una porción de filas de forma independiente.

Barridos de líneas (Líneas 109–153 y 161–202)

Los dos barridos alternos (`barrido_y` y `barrido_x`) son el núcleo del rendimiento:

- `!$omp parallel do`: crea la región paralela y reparte el bucle externo entre los hilos.
- `default(none)`: obliga a declarar explícitamente el alcance de cada variable, evitando errores de memoria compartida.
- **Variables compartidas (shared)**: las constantes físicas (`deltax`, `deltay`), la matriz de temperaturas (`tt`) y los vectores de frontera (`cfx`, `cfy`) son compartidos porque todos los hilos los leen y escriben en regiones bien delimitadas por sus índices.
- **Variables privadas (private)**: los arreglos del sistema tridiagonal de cada línea (`ax`, `bx`, `cx`, `rx`, `tx` y sus equivalentes en *y*) son locales a cada hilo. Cada hilo resuelve su propio sistema con la subrutina `tri` (líneas 142 y 190); si fueran compartidos, los hilos se sobrescribirían entre sí y el resultado sería incorrecto.

Cálculo del residuo (Líneas 206–215)

Para verificar convergencia se suma el cuadrado de las diferencias de temperatura entre iteraciones. Como es una acumulación en una sola variable escalar, se usa `reduction(+:residuo)`: cada hilo acumula su parte en una copia privada y OpenMP las combina al final, sin condiciones de carrera.

Exportación a VTK (Líneas 229–235)

La escritura del archivo VTK se hace con un solo hilo, en orden estricto por filas y columnas. Paralelizarla haría que los hilos escribieran en desorden, corrompiendo el archivo y haciéndolo ilegible para herramientas como gnuplot o ParaView.

4. Resultados

Para compilar y ejecutar el programa `calor2D` se usan los siguientes comandos

```
gfortran tridiagonal.f90 mod_utiles.f90 calor2D_paralelo.f90 -o
↵ calor2D
./calor2D
```

4.1. Solución estacionaria

Como se mencionó antes, los resultados de la simulación `calor2D` se guardaron en un archivo llamado `salida.vtk`, el cual puede ser visualizado fácilmente usando Paraview.

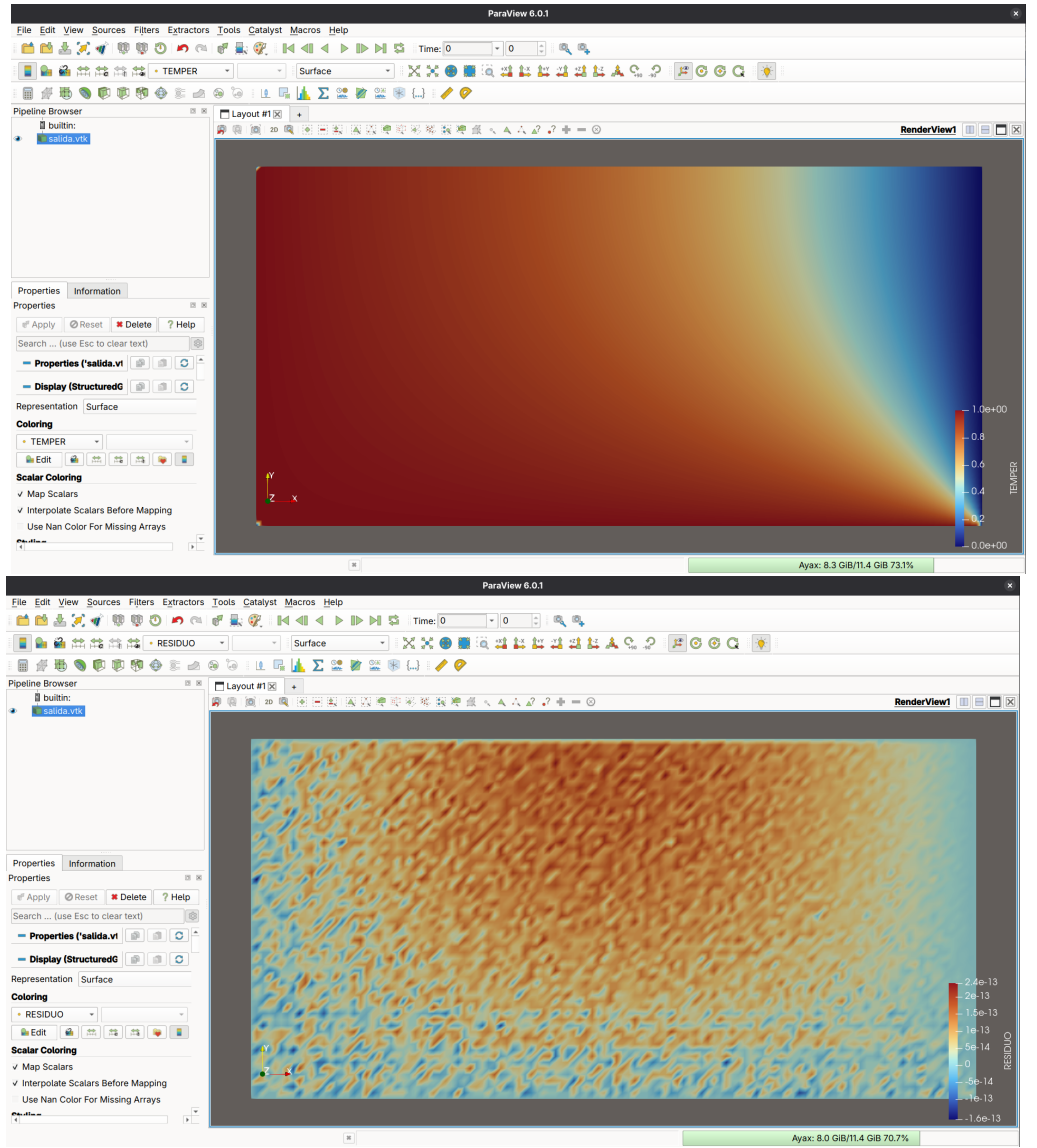


Figura 1: Gráficas de la solución estacionaria `tt(:, :)` y su residuo `resid_tt(:, :)` usando Paraview obtenidos de ejecutar el programa `calor2D` y leer el archivo `salida.vtk`.

En la figura 1 se observa claramente que se cumplen las condiciones de frontera establecidas; +1 en las fronteras izquierda e inferior, 0 en la frontera derecha y condición de Neumann en la frontera superior. También se observa que el residuo en cada nodo tiene valores de máximo 2.4×10^{-13} , lo que indica que nuestra solución discreta se acerca mucho a la teórica.

4.2. Rendimiento

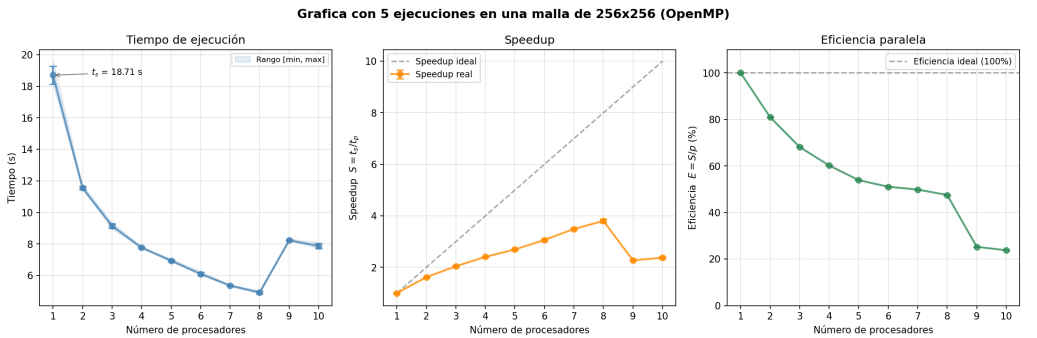


Figura 2: Rendimiento obtenido en promedio al ejecutar 5 veces `calor2D` y medir el tiempo de ejecución.

La gráfica 2 analiza el rendimiento de `calor2D` al aumentar la cantidad de procesadores de uno a diez usando `export OMP_NUM_THREADS`. En ella se observa que la velocidad del cálculo mejora de forma constante a medida

que se añaden hilos, pero encuentra su límite óptimo al alcanzar los ocho procesadores (porque es el número de hilos que mi CPU tiene de forma independiente). A partir de este punto, el rendimiento empeora de manera drástica debido a la saturación del sistema.

El análisis del tiempo de ejecución muestra que el programa requiere un máximo de 18.71 segundos cuando funciona de manera secuencial con un solo procesador. Al incrementar la potencia computacional, el tiempo disminuye progresivamente hasta registrar su punto más rápido con ocho procesadores, donde la tarea se completa en aproximadamente 5 segundos. Sin embargo, al emplear nueve y diez procesadores se eleva el tiempo de ejecución a casi 8 segundos.

La aceleración o *speedup* mide cuántas veces más rápido corre el programa en comparación con su ejecución básica. Aunque la expectativa ideal plantea una aceleración lineal equivalente al número de procesadores utilizados, el comportamiento real muestra que el programa alcanza un tope de velocidad de 3.8 veces más rápido con ocho procesadores, para luego caer en picada en los casos posteriores.

Finalmente, la eficiencia paralela refleja el porcentaje de aprovechamiento de cada procesador añadido. Este indicador disminuye desde el segundo procesador y se estabiliza cerca del 50 % entre los cinco y ocho núcleos. Al intentar utilizar nueve y diez procesadores, la eficiencia se desploma al 25 %, lo que demuestra un desperdicio de la capacidad del equipo. Este fenómeno confirma que el procesador físico empleado posee un límite de ocho núcleos, por lo que añadir hilos adicionales genera una congestión que ralentiza el trabajo.

5. Conclusiones

Se resolvió numéricamente la ecuación de calor estacionaria en 1D y 2D mediante diferencias finitas. En 1D, la matriz tridiagonal del sistema permitió aplicar directamente el algoritmo de Thomas que resuelve el problema de forma exacta (sin considerar el error de la aritmética de punto flotante) y secuencial. En 2D, el acoplamiento entre direcciones rompe la estructura tridiagonal estricta, pero el método de barridos alternos de líneas (ADI simplificado) recupera dicha estructura fila por fila y columna por columna, reaprovechando el mismo algoritmo de Thomas dentro de un esquema iterativo controlado por una tolerancia ε .

La paralelización con OpenMP resultó natural: cada línea define un sistema tridiagonal independiente, por lo que repartir los barridos entre hilos no introduce dependencias de datos, siempre que los arreglos de cada sistema se declaren de forma privada (`private`) y el residuo se acumule con `reduction(+:...)`.

El análisis de rendimiento confirmó la teoría: el tiempo bajó de 18.71 s en modo secuencial a ~ 5 s con ocho hilos, con un *speedup* máximo de $3.8\times$ y una eficiencia que se estabilizó cerca del 50 %. El punto óptimo coincidió exactamente con los ocho núcleos físicos del CPU; al forzar nueve o diez hilos, la saturación degradó el desempeño (eficiencia ~ 25 %). Esto evidencia que la ganancia paralela está acotada por el *hardware* disponible y que sobresuscribir los núcleos es contraproducente.

En conjunto, el trabajo muestra cómo un modelo físico sencillo sirve de banco de pruebas para combinar un algoritmo clásico eficiente (Thomas) con paralelismo de memoria compartida.

Referencias

[1] Toma, L. (2017). *Introduction to OpenMP*. Consultado el 5 de junio de 2026, desde <https://tildesites.bowdoin.edu/~ltoma/teaching/cs3225-GIS/fall17/Lectures/openmp.html>

A. Programa mod_utiles.f90

```
1 module utiles
2   !
3   implicit none
4   !
5   ! Iteradores y tamaño del problema
6   !
7   integer, parameter :: nx = 120, ny = 60, itermax=10000
8   !
9 contains
10  !
11  Subroutine residuo_temp(temp,deltax,deltay,resid)
12    !
13    ! Esta subrutina eval'ua el laplaciano de un campo escalar y devuelve
14    ! su elemento m'as grande
15    !
16    use omp_lib
17    !
18    implicit none
19    !
20    double precision, intent(in) :: temp(:,:)
21    double precision, intent(in) :: deltax,deltay
22    double precision, intent(out) :: resid(:,:)
23    !
24    integer :: sx, sy
25    integer :: ii, jj
```

```
26  !
27  ! -----
28  !
29  ! Se determinan las dimensiones del arreglo temp
30  !
31  sx = size(temp,1)
32  sy = size(temp,dim=2)
33  print*, "DEBUG: ", sx, sy
34  !
35  ! Se recorren los puntos de la malla
36  !
37  do jj = 2, sy-1
38    !
39    do ii = 2, sx-1
40      !
41      resid(ii,jj) = ( temp(ii-1,jj)-2.d0*temp(ii,jj) + &
42        & temp(ii+1,jj) )/(deltax*deltax)+&
43        & ( temp(ii,jj-1)-2.d0*temp(ii,jj) +&
44          temp(ii,jj+1) )/(deltay*deltay)
45    end do
46    !
47  end do
48
49 end Subroutine residuo_temp
50 !
51 !-----
52 !
53 ! Subrutina de postproceso de archivos vtk
54 !
55 subroutine postproceso_vtk(&
56   &xo,yo,tempo,residuo,archivoo&
57   &)
58   ! use malla, only : mi, nj, DBL, mic, njc, zkc
59   implicit none
60   integer :: i,j,k
61   double precision, dimension(nx), intent(in) :: xo
62   double precision, dimension(ny), intent(in) :: yo
63   double precision, dimension(nx,ny), intent(in) :: tempo, residuo
64   character(48), intent(in) :: archivoo
65   character(64) :: mico,njco,zkco
66   character(128) :: npuntosc
67   !
68   ! Creaci'on de cadenas de caracteres para el contenido de los archivos
69   !
70   write(mico,*) nx
71   write(njco,*) ny
72   write(zkco,*) 1
73   write(npuntosc,*) (nx)*(ny)*1
74   !
75   !*****
76   ! VTK
77   open(78, file = trim(archivoo), access='stream', convert="big_endian")
78
79   write(78) '# vtk DataFile Version 2.3'//new_line(' ')
80   write(78) '3D Mesh'//new_line(' ')
81   write(78) 'BINARY'//new_line(' ')
82   write(78) 'DATASET STRUCTURED_GRID'//new_line(' ')
83   write(78) 'DIMENSIONS '//'trim(mico)//trim(njco)//trim(zkco)//new_line('a')
84   write(78) 'POINTS '//'trim(npuntosc)//' float',new_line('a')
85   do k = 1, 1
86     do j = 1, ny
87       do i = 1, nx
88         write(78) real(xo(i)),real(yo(j)),0.0
89       enddo
90     enddo
91   end do
92   write(78) new_line('a')//'POINT_DATA '//'trim(npuntosc)
93   write(78) 'SCALARS TEMPER float',new_line('a')
94   write(78) 'LOOKUP_TABLE default',new_line('a')
95   do k = 1, 1
96     do j =1, ny
97       do i =1, nx
98         write(78) real(tempo(i,j))
99         ! write(78) real(bo(i,j))
100      end do
101    end do
102  end do
103  write(78) new_line('a')//'SCALARS RESIDUO float',new_line('a')
104  write(78) 'LOOKUP_TABLE default',new_line('a')
105  do k = 1, 1
106    do j =1, ny
107      do i =1, nx
108        write(78) real(residuo(i,j))
109      end do
110    end do
111  end do
112  ! write(78) new_line('a')//'VECTORS VELOCITY float',new_line('a')
113  ! do k = 1, 1
114  !   do j =1, nj+1
115  !     do i =1, mi+1
116  !       write(78) real(uo(i,j)),real(vo(i,j)),0.0
117  !     end do
118  !   end do
119  ! end do
120  close(78)
121  !
122  ! 100 FORMAT(3(f12.6));
123  ! 110 FORMAT(A);
124  ! 111 FORMAT(A,/);
125  ! 120 FORMAT(A,I4,I4,I4);
126  ! 130 FORMAT(A,I10,A);
127  ! 140 FORMAT(A,I10);
128  !
129 end subroutine postproceso_vtk
130
131 end module utiles
```